

Software Engineering (STE)

Course Code : 315323

Lecturer – Asad Shamna

Semester – 5th

I. RATIONALE

Software Engineering is the foundation for professional processes to be followed for designing, developing, testing and maintaining software involving principles, different techniques, and practices for software development. This course enable students to develop requisite abilities to follow systemic and disciplined approach to software development that aims to create high quality, reliable and maintainable software.

II. INDUSTRY / EMPLOYER EXPECTED OUTCOME

Apply software engineering principles to develop software product.

III. COURSE LEVEL LEARNING OUTCOMES (COS)

Students will be able to achieve & demonstrate the following COs on completion of course based learning

CO1 – Select suitable software development process model

CO2 – Prepare software requirement specification.

CO3 – Construct different Software design models

CO4 – Apply different planning and cost estimation techniques for a software product

CO5 – Apply project management techniques in software development.

CO6 – Use quality assurance principles in software development

UNIT - I SOFTWARE DEVELOPMENT PROCESS [12M]

1.1 Software characteristics, Types of software.

**1.2 The Process: Software Engineering: A Layered approach –
Process, Methods and Tools**

1.3 Software development framework.

1.4 Software Process Model: Waterfall Model

1.5 Incremental Process Model: RAD Model

1.6 Evolutionary Process Models: Prototyping model, Spiral model

1.7 Agile Process Model: Extreme Programming, Adaptive Software Development (ASD), Scrum, Dynamic System Development Method (DSDM), CRYSTAL. Agile Unified Process (AUP)

1.1 Software characteristics, Types of software.

What is Software?

- Software is a set of instructions used to operate computers and perform specific tasks.
- It is the opposite of hardware, which is the physical aspect of a computer.
- It is not a physical part of the computer but is essential for it to function.

What is Software Engineering?

Software Engineering is the systematic approach to the design, development, testing, deployment, and maintenance of software. It applies engineering principles to software creation to ensure:

- i. High quality
- ii. Low cost
- iii. On-time delivery
- iv. Maintainability
- v. Scalability

“Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.”

Characteristics of Software

- Software is developed or engineered; it is not manufactured.
- Software is developed using programming languages and requires continuous updates and maintenance.
- Software can be easily changed or updated according to user needs.
- Software systems can be highly complex with many interconnected modules.
- Unlike hardware, software does not degrade over time but may become outdated or may contains unusual bugs.
- Software needs hardware to run but can work across various hardware types with modifications.

TYPES OF SOFTWARE

SYSTEM SOFTWARE



Manages computer hardware and provides a platform for running applications.

Examples: Windows, Linux, Device Drivers

APPLICATION SOFTWARE



Helps users perform specific tasks.
Examples: MS Word, Excel, Chrome, WhatsApp, VLC

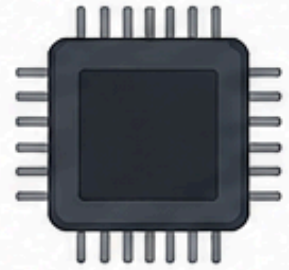
EMBEDDED SOFTWARE



Built into devices to control their specific functions.

Examples: Washing Machine, Car, Microwave Oven

FIRMWARE



Low-level software programmed into hardware devices.

Examples: BIOS, Router Firmware, SSD Firmware

NETWORKING SOFTWARE



Helps in managing and communicating over computer networks.

Examples: Wireshark, Cisco Packet Tracer

SECURITY SOFTWARE



Protects systems and data from threats, viruses, malware, and unauthorized access.

Examples: Antivirus, Firewall, Malware Scanner

GAMING SOFTWARE



Software designed for entertainment and playing games.

Examples: PUBG, GTA, Call of Duty

CLOUD SOFTWARE



Runs on remote servers and accessed via the internet.

Examples: Google Drive, Office 365, Dropbox

DEVELOPMENT SOFTWARE

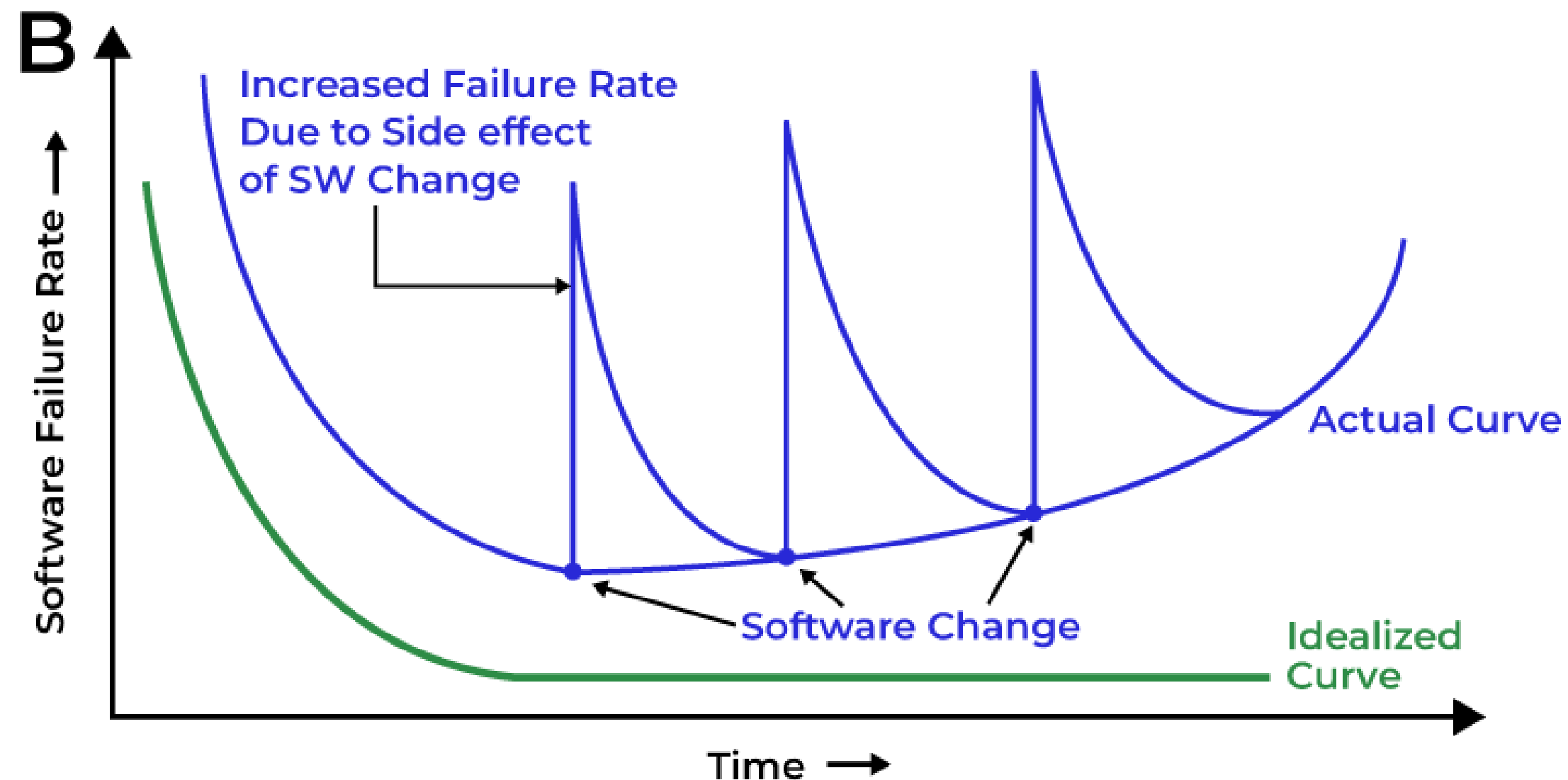


Used by developers to create, test, and maintain software.

Examples: VS Code, IntelliJ IDEA, Eclipse

Failure Curve

- The ideal goal is to achieve the "**idealized curve**" – low and stable failure rate.
- But in reality, changes often increase failure rate unless:
 - Proper testing, documentation, and change control are followed.
- This graph highlights the importance of careful maintenance and regression testing in software engineering.



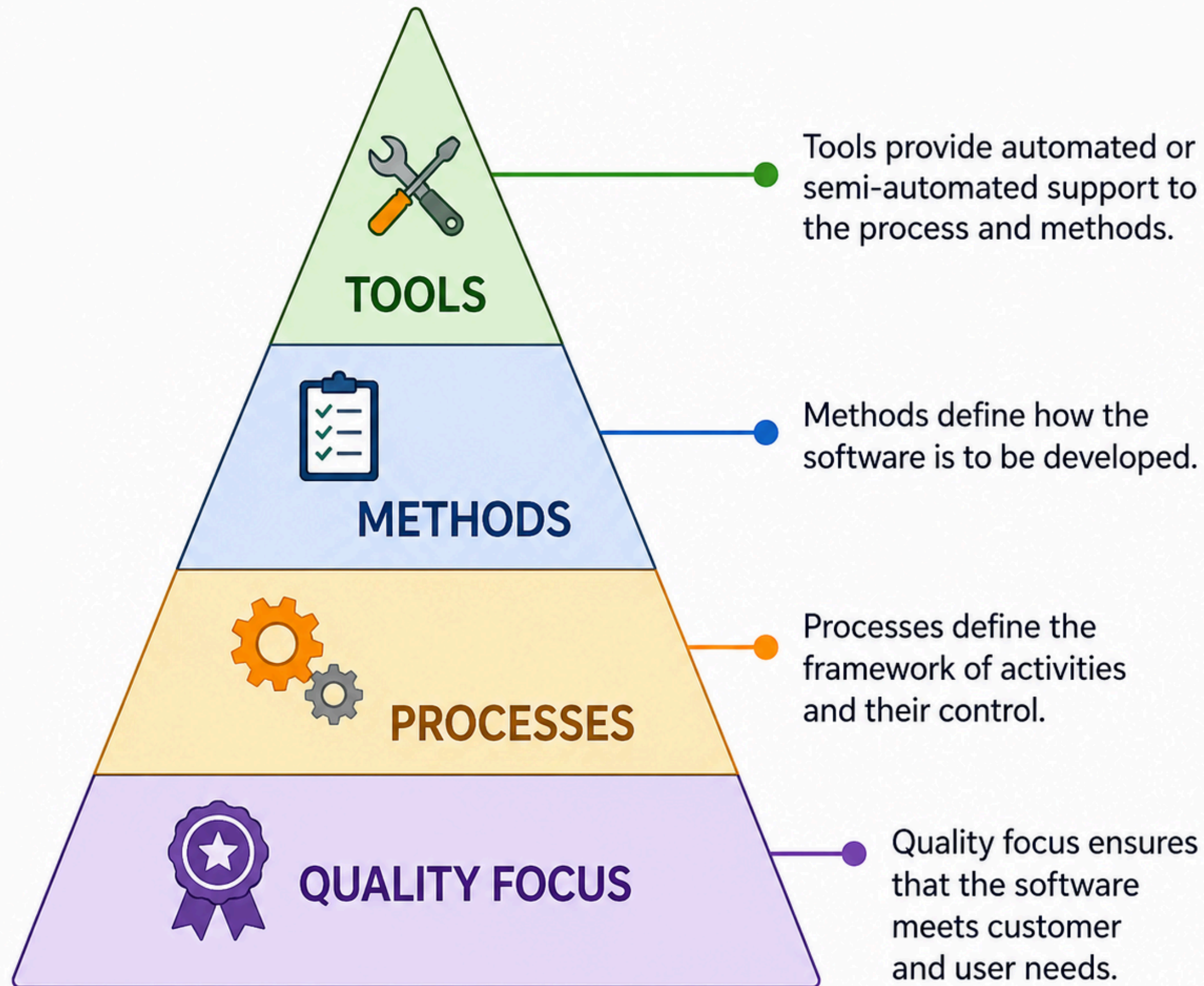
1.2 The Process: Software Engineering: A Layered approach - Process, Methods and Tools.

The Process: Software Engineering - A Layered Approach

- Software Engineering uses a layered technology approach.
- It means that to develop quality software, several layers work together.

Tools
Methods
Processes
Quality Focuses

Layered Approach of Software Process

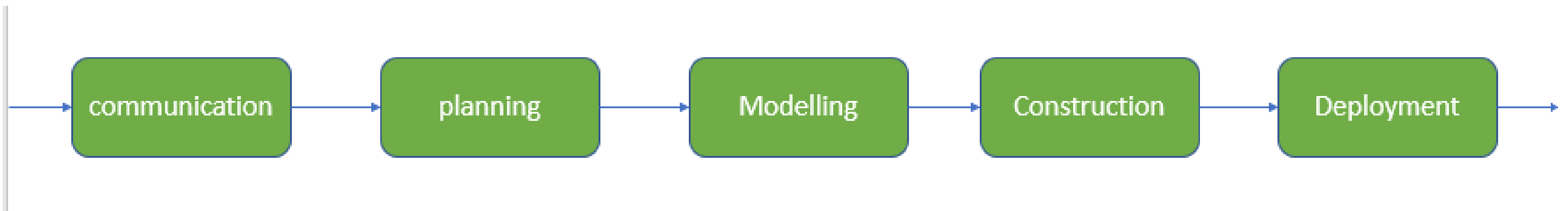


1) Quality Focus:

- Customer Satisfaction depends on the quality of the application.
- Goal: Ensure high-quality software. (Developing high quality Software)
- It act like a bedrock that support Software Engineering.
- "Bedrock" means the most important, fundamental support on which everything else is built.

2) Process Layer:

- It is foundation for software Development.
- Defines the framework for managing all activities.
- It act as a glue that holds all the layers together and enable timely development.
- Includes stages like:
 - Requirement analysis
 - Planning
 - Design / Modelling
 - Coding / Development / Construction
 - Testing
 - Deployment
 - Maintenance



3) Methods:

- Provides technical tasks to build software.
- Covers:
 - Analysis (e.g., data flow, requirement gathering)
 - Design (e.g., UI design, system architecture)
 - Coding principles
 - Testing techniques

4) Tools Layer

- Software tools automate and support methods.
- It is used to support process and methods.
- It provide automate and semi automated support when tools are integrated so that information created by one tools can be used by other.
- Examples:
- Visual Studio, Android Studio (coding)
- Selenium (testing)
- Git (version control)
- Figma, Adobe XD (design, prototype)
- MySQL, PostgreSQL, MongoDB (database)

1.3 Software development framework.

Software Development Framework

- A software development framework is a predefined structure used by developers to build and deploy applications efficiently.
- It provides reusable code, tools, libraries, and best practices to streamline development.

Definition:

- A **software development framework** is a platform for developing software applications. It provides a foundation with predefined classes, functions, and components to build applications faster, more securely, and with fewer errors.

TOP SOFTWARE DEVELOPMENT FRAMEWORKS

Pre-built structures and tools that make development faster,
easier and more organized.

The Django logo, featuring the word "django" in a lowercase, bold, green sans-serif font.

Django

Python

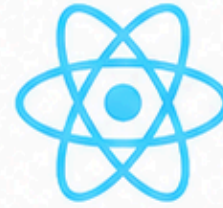
High-level Python
web framework
that encourages
rapid development
and clean design.



Spring

Java

Popular Java
framework for
building enterprise
applications with
robust features.



React

JavaScript

JavaScript library
for building user
interfaces, especially
single-page
applications.



Angular

TypeScript

Platform for building
mobile and desktop
web applications
with TypeScript.



Vue.js

JavaScript

Progressive JavaScript
framework for
building interactive
user interfaces.



ASP.NET Core

C#

Modern, cross-platform
framework by
Microsoft for building
web applications
and APIs.



Laravel

PHP

Elegant PHP
framework for
web artisans,
focused on simplicity
and productivity.



Ruby on Rails

Ruby

Convention over
configuration
framework for
building web
applications in Ruby.

express

Express.js

JavaScript (Node.js)

Fast, minimal and
flexible Node.js
web framework for
APIs and web
applications.



Flask

Python

Lightweight Python
web framework for
small to medium
sized applications
and APIs.

Key Features of Software Development Framework:

- **Code reusability** (prebuilt functions/components)
- **Standardization** (follows structured conventions)
- **Efficiency** (saves time and reduces bugs)
- **Integration** with other tools and libraries
- **Security** (built-in protection mechanisms)
- **Maintainability** (modular and clean code)

Advantages:

- Faster development process
- Fewer bugs and errors
- Easier maintenance and updates
- Better collaboration among teams
- Scalability and flexibility

Disadvantages:

- Learning curve
- May include unused features (bloat)
- Less flexibility for unique requirements
- Dependence on third-party updates

1.4 Software Process Model

A Software Process Model is a framework that defines the process and steps involved in developing a software product, from requirement gathering to final delivery and maintenance.

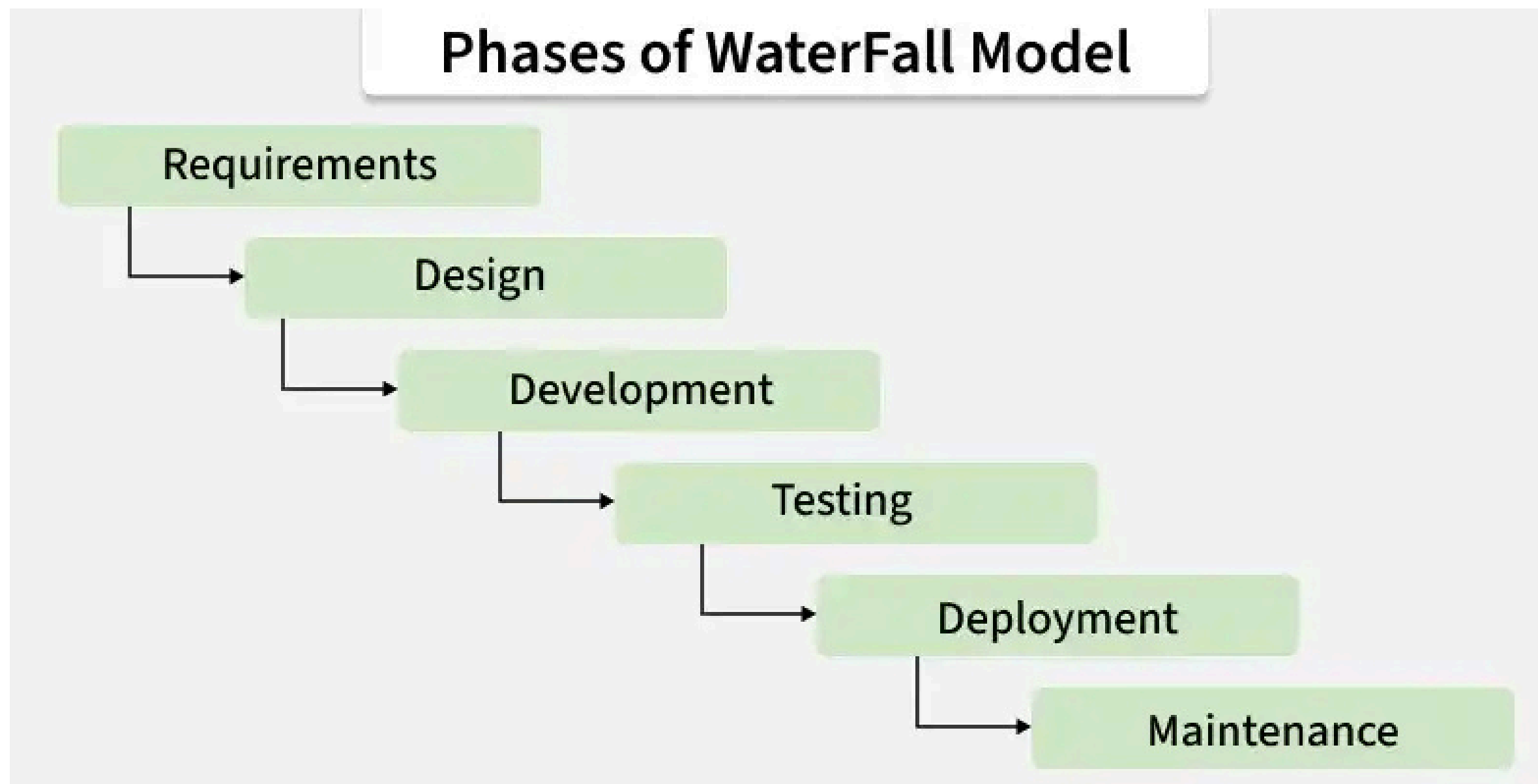
Phases in Any Software Process Model:

- Requirement Analysis
- Planning
- System Design
- Implementation (Coding)
- Testing
- Deployment
- Maintenance

“A software process model is an abstraction of the actual process, which is being described. It can also be defined as a simplified representation of a software process. Each model represents a process from a specific perspective.”

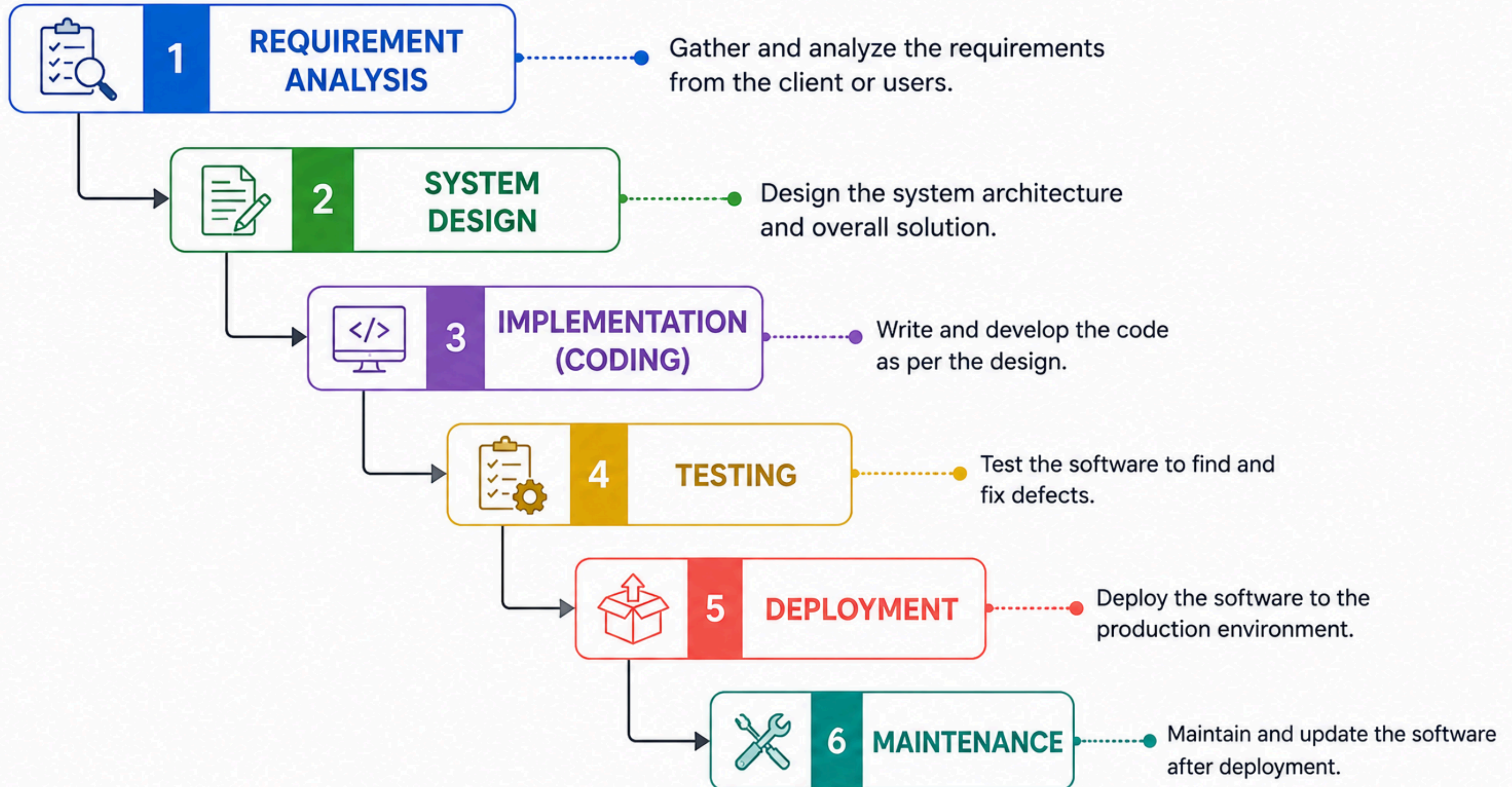
Waterfall Model

- The Waterfall Model is a linear and sequential software development process in which each phase must be completed before the next phase begins.
- It flows step-by-step like a waterfall, from top to bottom.
- The waterfall model is a traditional method, sometimes called the classic life cycle.
- This is one of the initial models.
- As the figure implies stages are cascaded and shall be developed one after the other.
- In other words one stage should be completed before the other begins.



WATERFALL PROCESS MODEL

A linear and sequential approach to software development.



Advantages of Waterfall Model

- Simple and easy to understand.
- Easy to manage due to its structured approach.
- Each phase has clear deliverables and goals.
- Documentation is maintained at every step.
- Best suited for small projects with fixed requirements.
- Easy to track project progress.

Disadvantages of Waterfall Model

- Difficult to make changes once a phase is completed.
- Not suitable for complex or long-term projects.
- Testing is done late, so errors are found late.
- Customer gets to see the product only at the end.
- High risk if initial requirements are misunderstood.
- No working software until the last phase.

1.5 Incremental Process Model

The Incremental Model is a process where the software is developed and delivered in small parts or increments, with each increment adding new features.

Key Points for Incremental Process Model:

- Requirements are divided into multiple standalone modules. (Login Page, Dashboard Page, Home Page, etc are divided into standalone modules)
- Each module is designed, developed, and tested separately.
- Each increment is integrated with the previous one.
- Final product is built step-by-step.

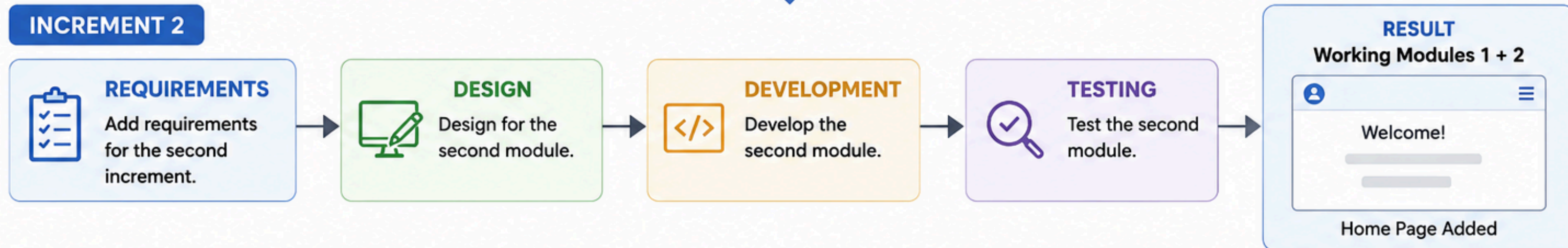
INCREMENTAL PROCESS MODEL

The software is developed in small increments. Each increment adds functional modules to the system.

INCREMENT 1



INCREMENT 2



INCREMENT 3



Each increment delivers a usable part of the system.
Final Result: Complete software with all modules integrated.

COMPLETE SOFTWARE

Advantages of Incremental Model

- Easier to test and debug smaller parts.
- Delivers working software early.
- Customer feedback is possible after each increment.
- Easier to manage risk.

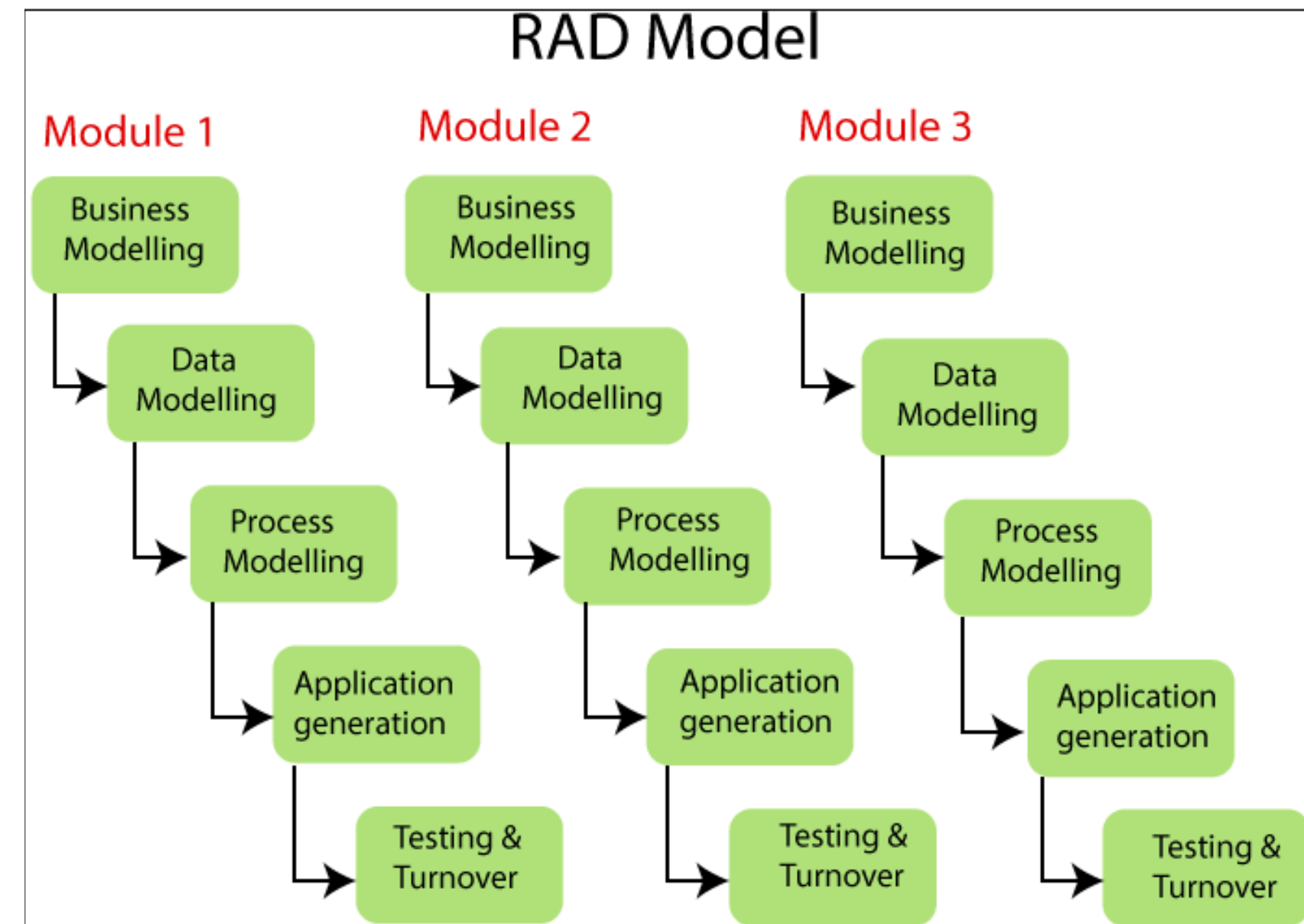
Disadvantages of Incremental Model

- Needs good planning and design upfront.
- Not ideal if requirements change frequently.
- Integration of multiple increments can be complex.

1.5 Incremental Process Model: RAD (Rapid Application Development) Model

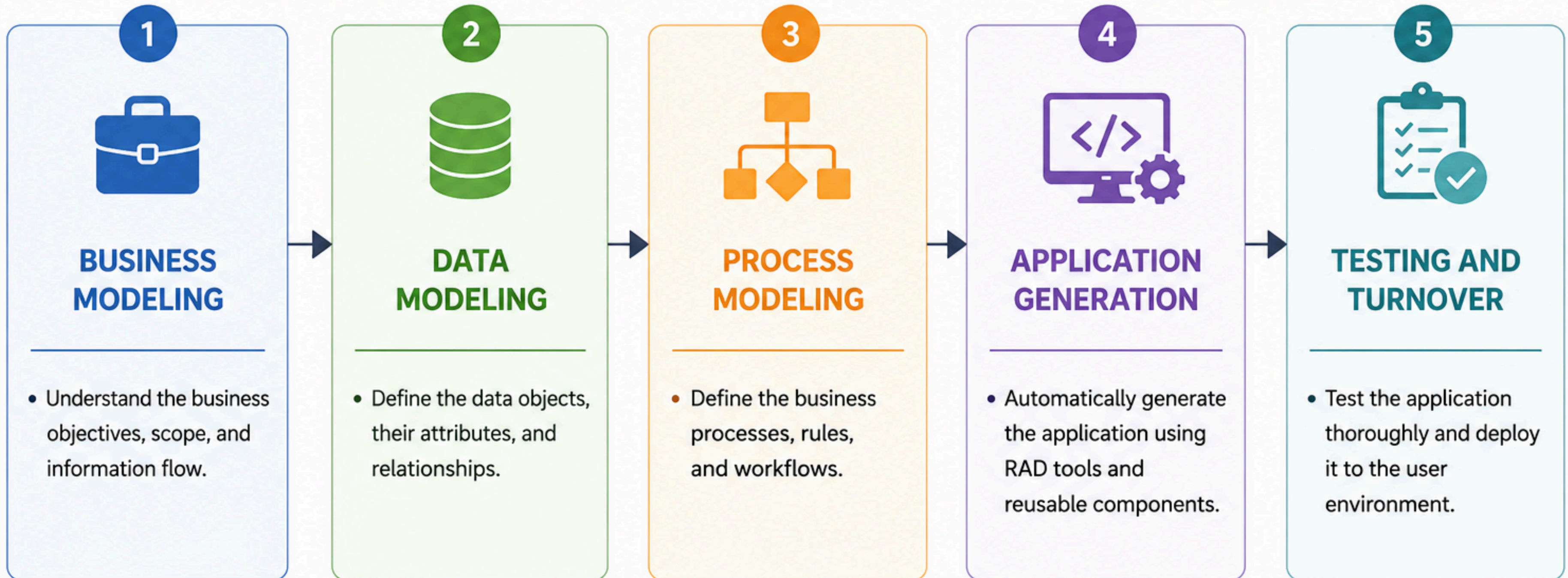
Rapid Application Development Model (RAD)

- The RAD Model focuses on rapid prototyping and quick feedback over long development and testing cycles.
- It aims to develop software quickly using reusable components.
- Phases:
 - a. Business Modeling
 - b. Data Modeling
 - c. Process Modeling
 - d. Application Generation
 - e. Testing and Turnover



RAD MODEL – PHASES

RAD (Rapid Application Development) is an iterative process model that emphasizes fast development and customer involvement.



1) Business Modelling

- The business model for the product under development is designed in terms of flow of information and the distribution of information between various business channels.
- Understand how the business works.
- Identify the flow of information (who does what, when, and how).
- Example: A school wants a system — we find out how admissions, fees, and exams work.

2) Data Modelling

- The information gathered in the Business Modelling phase is reviewed and analyzed to form sets of data objects vital for the business.
- Find out what data the business uses.
- Organize data into tables, fields, and relationships.
- Example: Student info, course list, marks — how they are connected.

3) Process Modelling

- The data object sets defined in the Data Modelling phase are converted to establish the business information flow needed to achieve specific business objectives as per the business model.
- Define how the data is processed and how it flows.
- Show how input becomes output using process diagrams.
- Example: When a student enrolls, their data is stored, and a fee slip is generated.

4) Application Generation

- The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.
- Use tools to build the actual software (often with reusable components).
- Code is quickly created with the help of RAD tools or automation.

5) Testing & Turnover

- Test each part as soon as it is built.
- Since modules are small and fast, testing is quick.
- After testing, the software is handed over to the client
- The overall testing time is reduced in the RAD model as the prototypes are independently tested during every iteration.
- However, the data flow and the interfaces between all the components need to be thoroughly tested with complete test coverage.
- Since most of the programming components have already been tested, it reduces the risk of any major issues.

Advantages of RAD Model

- Faster development and delivery.
- Encourages customer feedback and involvement.
- Reusability of components saves time.
- Reduces development and testing time.

Disadvantages of RAD Model

- Not suitable for large or complex projects.
- Requires highly skilled developers.
- Depends on strong team collaboration.
- Poor performance if system components are not reusable.

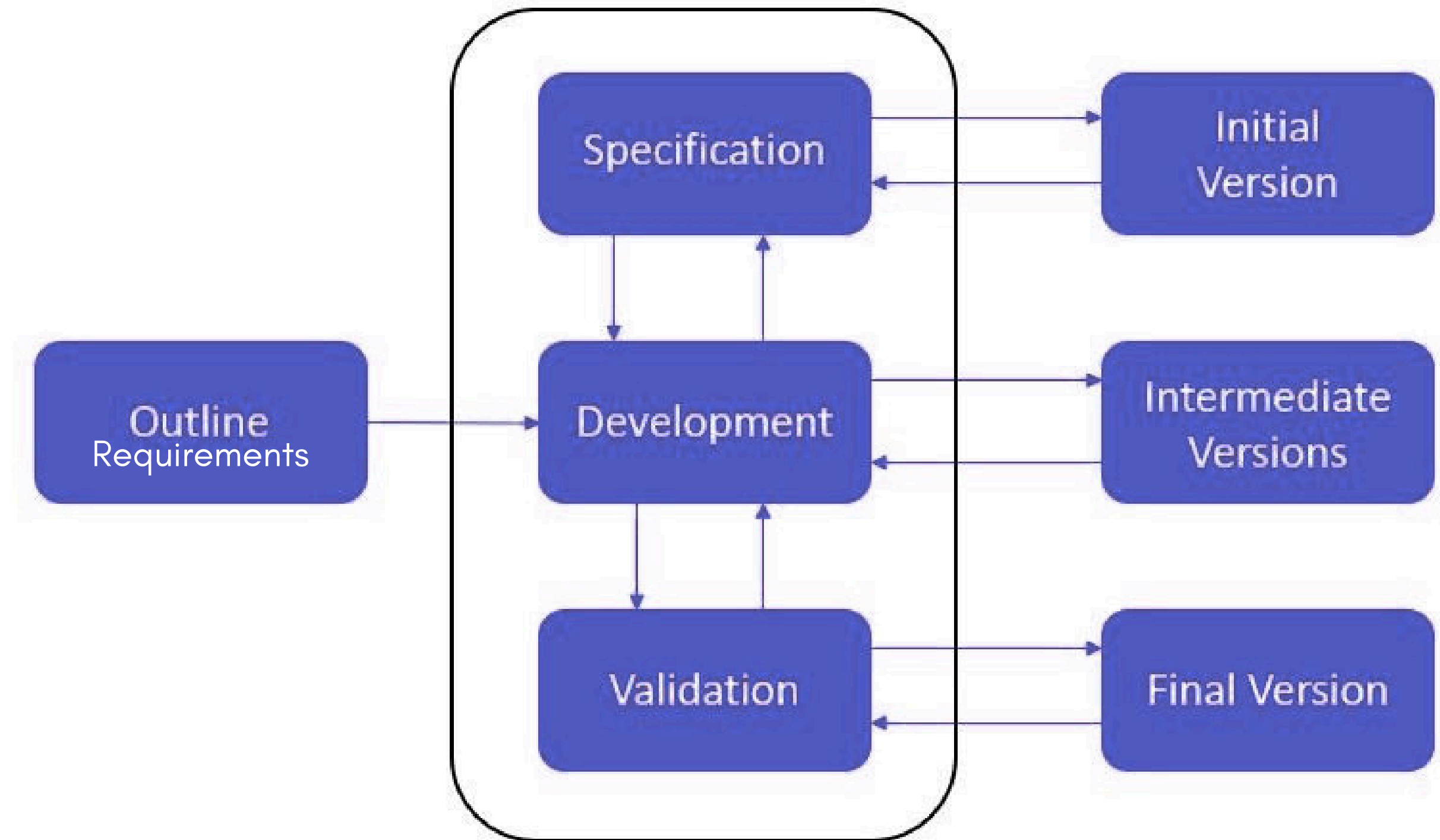
1.6 Evolutionary Process Models: Prototyping model, Spiral model

Evolutionary Process Models

- The evolutionary model is based on the concept of making an initial product and then evolving the software product over time with iterative and incremental approaches with proper feedback.
- In this type of model, the product will go through several iterations and come up when the final product is built through multiple iterations.
- The development is carried out simultaneously with the feedback during the development.
- This model has a number of advantages such as customer involvement, taking feedback from the customer during development, and building the exact product that the user wants.
- Because of the multiple iterations, the chances of errors get reduced and the reliability and efficiency will increase.

1.6 Evolutionary Process Models: Prototyping model, Spiral model

Evolutionary Process Models



Prototyping model

- Prototyping Model is a software development model in which a working prototype is created first, evaluated by the customer, and refined based on feedback until the requirements are understood and accepted.
- A prototype (basic working model) of the software is created quickly to understand the requirements.
- This is shown to the user to get feedback. Based on the feedback, the final system is developed.

Phases of Prototyping Model

- Prototyping Model has six phases as follows:
 - Requirements gathering and analysis
 - Quick design
 - Build a Prototype
 - Initial user evaluation
 - Refining Prototype
 - Implement Product and Maintain

Six Phases of Prototyping Model

1. Requirements Gathering and Analysis

- Collect and understand the user's requirements.

2. Quick Design

- Create a basic design of the system.

3. Build a Prototype

- Develop a simple working model of the system.

4. Initial User Evaluation

- Show the prototype to the user and collect feedback.

5. Refining the Prototype

- Modify and improve the prototype based on feedback.

6. Implement Product and Maintain

- Develop the final product, test it, deploy it, and maintain it.

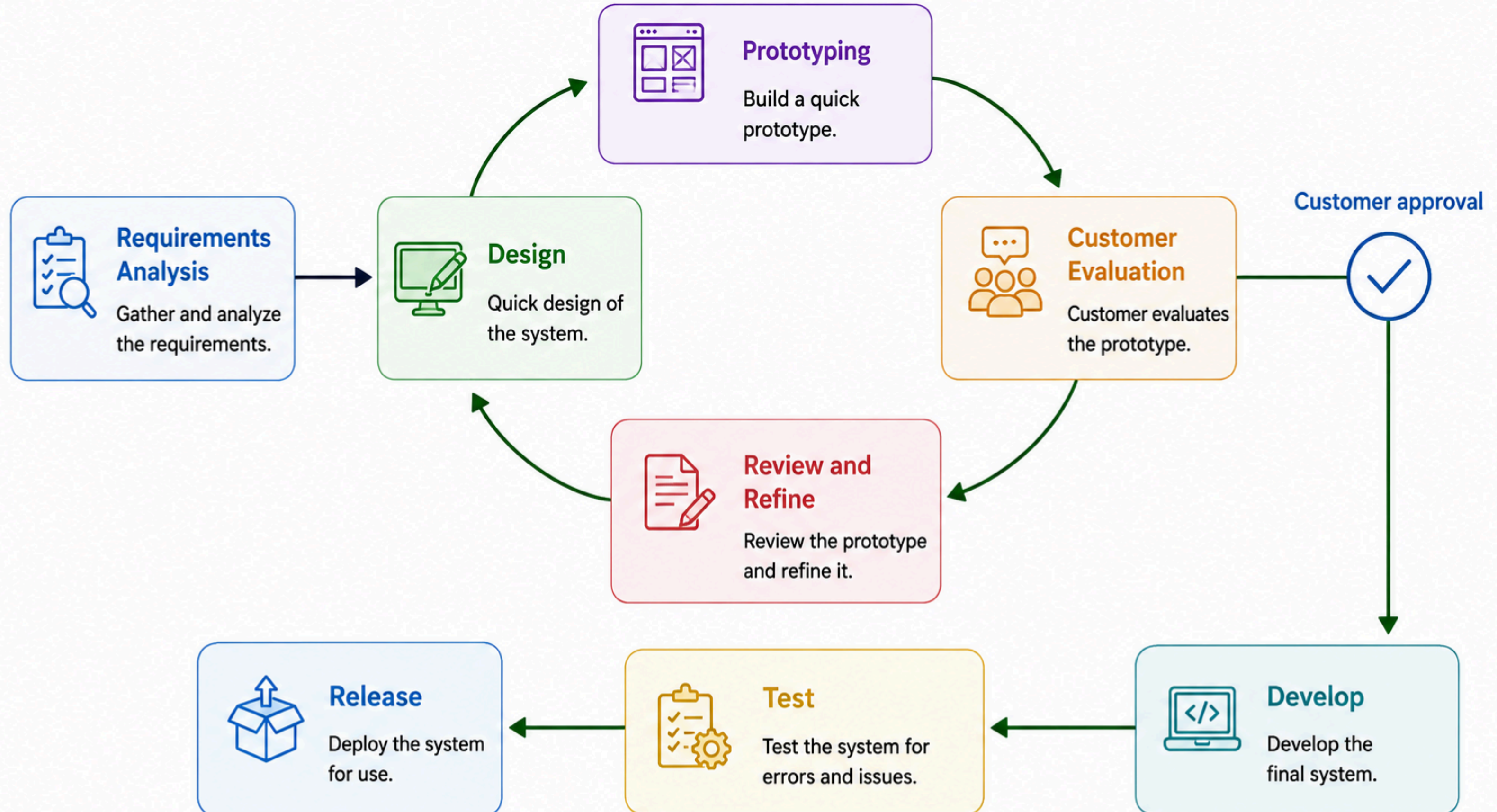
- **Advantages of Prototype Model:**

- Early feedback from users
- Better understanding of requirements
- Reduces chances of major failure

- **Disadvantages of Prototype Model:**

- May cause confusion between prototype and final product
- Not suitable for large or complex systems

PROTOTYPE MODEL



Spiral Model

- The Spiral Model is a risk-driven software development model in which software is developed through repeated cycles called spirals. Each cycle includes planning, risk analysis, development, testing, and customer review.
- Combines features of prototyping and waterfall model.
- It focuses on risk analysis and develops the software in spiral (cyclical) phases.

Phases in Each Spiral:

1. Planning – Requirements are gathered
2. Risk Analysis – Identify and solve potential risks
3. Engineering – Develop and test the product
4. Evaluation – Get feedback from customer

Each loop of the spiral results in a more refined version of the software.

Four Phases of the Spiral Model

1. Objective Setting (Planning)

- Project goals and requirements are identified
- Functional and non-functional requirements are analyzed
- Possible solutions are explored

2. Risk Analysis

- Risks related to cost, schedule, performance, and technology are identified
- Best solution is selected
- Prototypes are built to reduce risks

3. Development and Testing

- Selected features are designed, developed, and tested
- The next working version of the software is created

4. Review and Planning

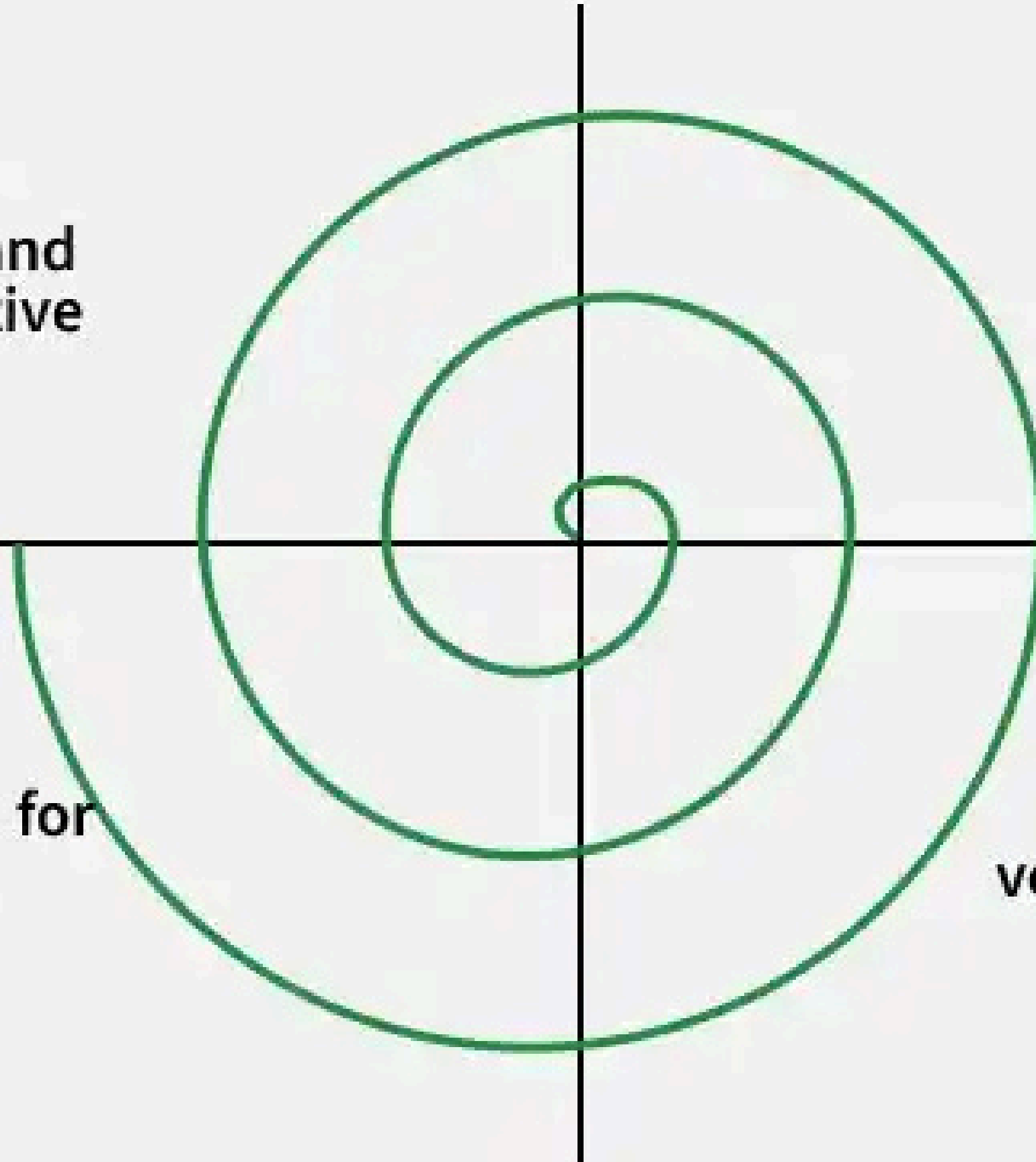
- Customers evaluate the current version
- Feedback is collected
- Planning for the next iteration begins

**1. Objectives
determination and
identify alternative
solutions**

**2. Identify and
resolve Risks**

**3. Develop next
version of the Product**

**4. Review and plan for
the next Phase**



Real Software Example: E-Commerce Website

Consider developing an Amazon-like e-commerce website. The software can be developed in multiple spirals:

Spiral 1:

Requirements → Product listing + Prototype → Identify risks → Customer review

Spiral 2:

Develop → User registration + Shopping cart → Test → Review

Spiral 3:

Develop → Secure payment + Order tracking → Test security and performance → Review

Spiral 4:

Develop → Search + Reviews → Final testing → Deployment

This example follows the Spiral Model because the system is built gradually while risks are identified and feedback is collected in every cycle.

Advantages of Spiral Model:

- Excellent for large, high-risk projects
- Risk is evaluated at every stage
- Software improves with each cycle

Disadvantages of Spiral Model:

- Expensive and time-consuming
- Requires expertise in risk analysis

1.7 Agile Process Model: Extreme Programming, Adaptive Software Development (ASD), Scrum, Dynamic System Development Method (DSDM), CRYSTAL. Agile Unified Process (AUP)

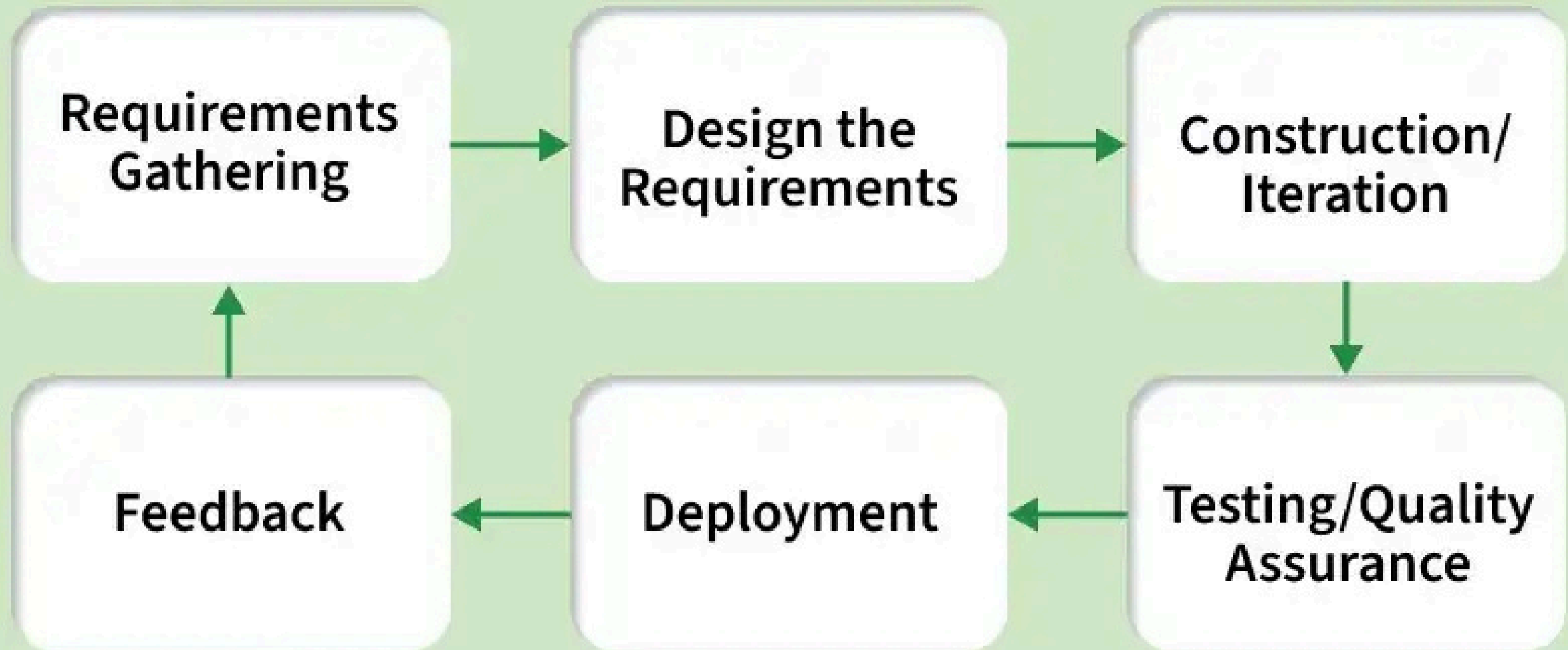
Agile Process Model

Agile is a software development model that combines both iterative (repeat) and incremental (step-by-step) methods.

The main focus is on:

- Customer satisfaction
- Quick delivery
- Flexibility to change
- The product is divided into small parts (called builds).
- These builds are made and delivered in short time periods called iterations (usually 1 to 3 weeks).
- After each iteration, the software is shown to the customer for feedback and improvements.

Steps in Agile SDLC Model



Steps in the Agile Model

The Agile Model is a combination of iterative and incremental process models. The phases involve in Agile SDLC Model are:

1. Requirement Gathering

In this step, the development team must gather the requirements, by interaction with the customer. Development team should plan the time and effort needed to build the project. Based on this information they can evaluate technical and economical feasibility.

- Meet with the customer to really understand their needs and what they expect from the software.
- Identify requirements and align them with business objectives so everyone shares the same understanding.
- Estimate how much time and effort it will take to develop the software.
- Assess if the project is technically possible and whether it's worth the investment from both a technical and economic standpoint.

2. Design the Requirements

- After gathering requirements, the next step is to design the overall structure of the system based on those needs. This ensures the software is organized in a way that meets user expectations.
- Wireframes are then created for the user interface. These act as simple layouts that show how the software will look and how users will interact with it, making sure it is easy to use.
- At this stage, high-level designs using UML (Unified Modeling Language) diagrams are prepared to visually show the system structure and how different components are connected.
- Prototypes are also developed to give stakeholders an early view of the system. This helps in collecting feedback early and making necessary changes before full development begins.

3. Construction / Iteration

In this step, the development team starts working on the project with the goal of delivering a working product. Each cycle typically consists of 1-4 weeks, and at the end of each iteration, a working version of the software is delivered.

- The team develops the features identified during the requirement and design phases.
- New functionalities are coded and integrated into the software based on the goals of that specific iteration.
- After each cycle, a usable version of the software is ready for review or use.
- With every iteration, the software is improved by adding new features and refining existing ones.

4. Testing / Quality Assurance

In this phase, the software is tested to ensure it works correctly and meets the required standards. The team identifies and fixes bugs, errors, and performance issues to improve overall quality.

- Unit Testing: Checks small pieces of code to ensure each part of the program works correctly on its own. It focuses on testing individual units or components.
- Integration Testing: Verifies that different units of the software work properly when combined and helps identify issues in their interaction.
- System Testing: Ensures that the complete software meets user requirements and works correctly in all possible scenarios.

5. Deployment

In this step, the development team deploys the working software to end users. Once an iteration is completed and fully tested, the software is ready for release.

In Agile, deployment is not a one-time activity; it is a continuous process where updates and improvements are released regularly to keep the software evolving.

- Deploy the software to a test or live environment so that it can be used by customers or end users
- Make the software accessible to users and ensure they can start using it as expected.
- Ensure the deployment runs smoothly and quickly fix any issues that arise.

6. Feedback

This is the final step of the Agile Model, where the team collects feedback on the product and works on fixing issues based on input from customers.

- Take feedback from customers, users, and stakeholders after each iteration.
- The team analyzes how well the product meets user needs and identifies areas for improvement.
- Check for bugs or issues that need fixing.
- Make adjustments to the development plan based on feedback to improve the product further.

Methodologies of Agile Process Model

- The Agile thought process had started early in the software development and started becoming popular with time due to its flexibility and adaptability.
- The most popular Agile methodologies include,
 - Scrum (1995)
 - Crystal Clear
 - Extreme Programming (1996)
 - Adaptive Software Development (ASD)
 - Dynamic Systems Development Method (DSDM) (1995).
 - Agile Unified Process
- These are now collectively referred to as Agile Methodologies, after the Agile Manifesto was published in 2001.

Methodologies of Agile Process Model

Extreme Programming (XP) :

Extreme Programming (XP) is one of the most popular Agile software development methodologies focused on delivering high-quality software quickly and efficiently.

- Promotes close collaboration between developers, testers, and customers.
- Supports iterative development with faster delivery and continuous improvement.
- XP is a way to build software quickly and with good quality.
- Two developers work together on one computer (this is called pair programming).
- Before writing the actual code, they first write tests to check if the code works (test-driven development).
- The team gets feedback from the customer every day, so they can make changes quickly.
- Best for: Projects where requirements change often.

The Extreme Programming Life Cycle consists of five phases:

1. Planning: In this stage, customers define their requirements as user stories. The team estimates the effort for each story and plans releases based on priority.
2. Design: Only the necessary design for current needs is created. A simple and shared understanding of the system is maintained to keep the design clear and manageable.
3. Coding: XP encourages pair programming, where two developers work together to improve code quality. Tests are written before coding (TDD), and code is frequently integrated into a shared repository.
4. Testing: Strong focus is given to testing, including unit tests and acceptance tests. Unit tests verify individual features, while acceptance tests ensure the system meets customer requirements.
5. Listening: Continuous customer feedback is taken to understand requirements better and adapt to changes effectively.

Adaptive Software Development (ASD):

Adaptive Software Development (ASD) is a software development methodology that emphasizes adaptability to respond to changing requirements and environments. It focuses on continuous learning and is considered a part of agile software development.

1. Iterative Development: ASD breaks down the project into small, manageable iterations, delivering incremental values.
2. Risk Management: ASD involves identifying and addressing risks early in the development process.
3. Continuous Testing: It integrates testing throughout the development process to ensure high-quality outcomes.
4. Decentralized Control: It encourages decision-making at the team level instead of relying on top-down directives.
5. Customer Satisfaction: It prioritizes customer satisfaction by delivering products that meet their needs and expectations.

Adaptive Software Development (ASD):

- ASD is a method of making software that helps the team to change plans easily, learn as they go, and work better together.
- It is used when the project is not clearly planned from the start or when things keep changing.

Three Simple Steps:

1. Speculate – Make a plan (even if it's not perfect)
2. Collaborate – Work together as a team
3. Learn – Use feedback to improve the next step

Best for: Projects where requirements are unclear or keep changing

Adaptive Software Development (ASD) – Three Phases

1. Speculation

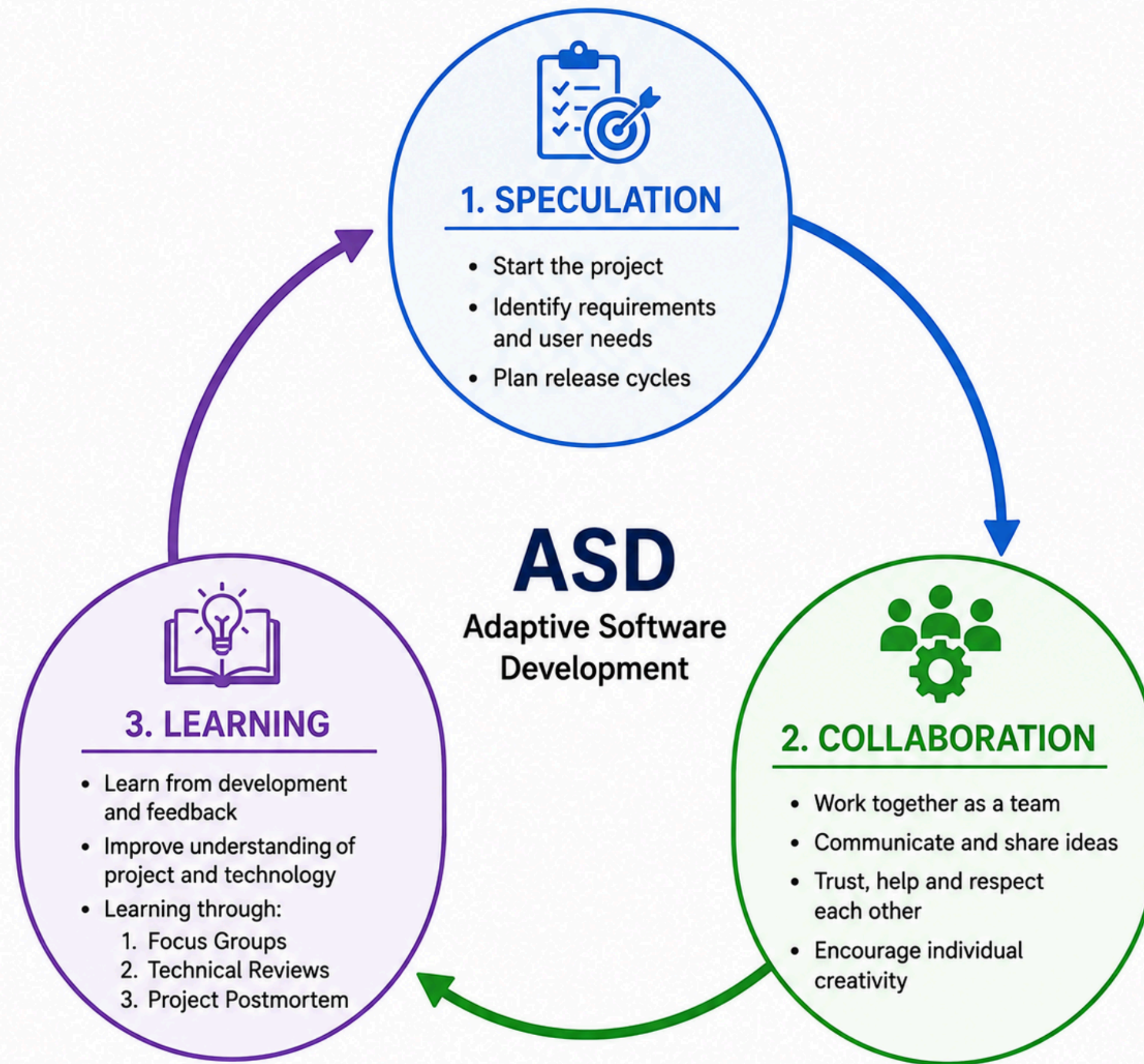
- Start the project and prepare the initial plan.
- Identify requirements, user needs, and project goals.
- Divide the project into release cycles.

2. Collaboration

- Encourage teamwork and communication.
- Team members share ideas and solve problems together.
- Members should trust, help, and respect each other.
- Individual creativity is also encouraged.

3. Learning

- Team members learn from development and feedback.
- Improves their understanding of the project and technology.
- Learning is done through:
 - a. Focus Groups – Get user feedback.
 - b. Technical Reviews – Review technical work.
 - c. Project Postmortem – Learn from completed work.



Flow: Speculation → Collaboration → Learning

Scrum:

Scrum is a way to build software by working in small time periods called sprints (usually 1-4 weeks).

Team Roles:

1. Product Owner – Decides what features to make
2. Scrum Master – Makes sure the team works smoothly
3. Team – Builds the product

Daily Stand-up:

- A short meeting every day
- Team members say:
 - What they did
 - What they will do
 - Any problems they have

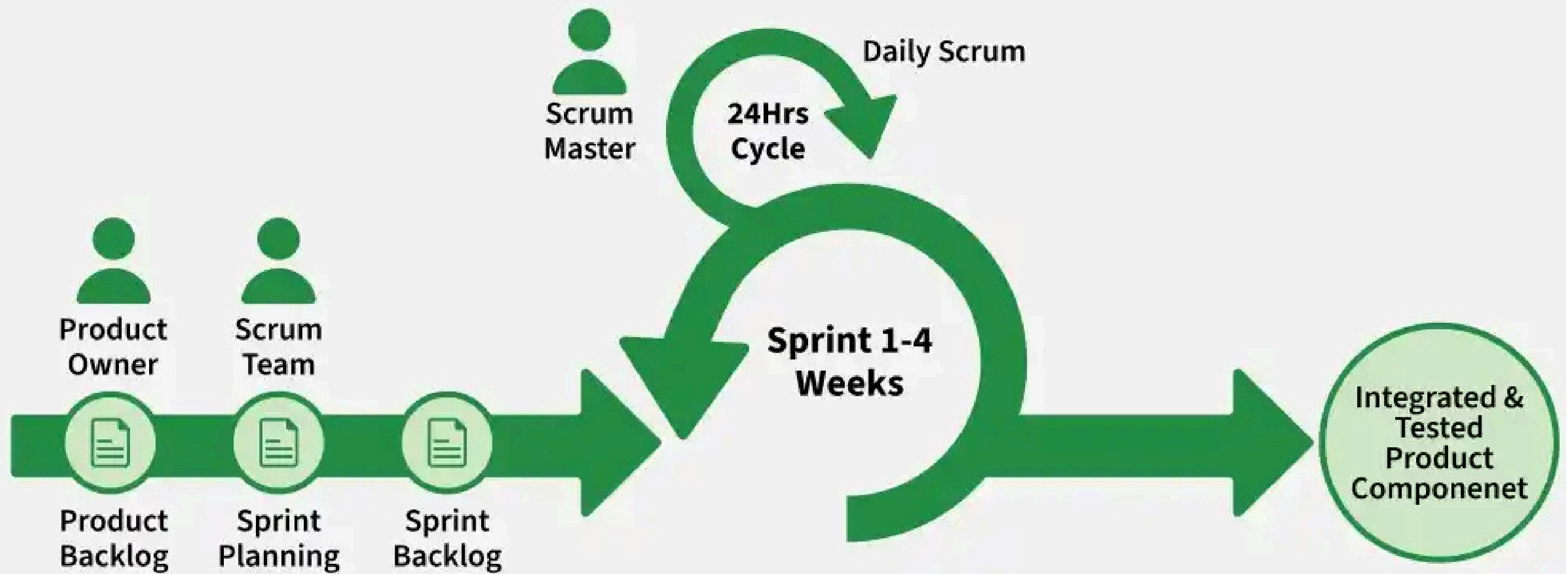
Best for: Team projects where work is done in steps and needs regular updates.

Scrum:

Scrum is an Agile framework used to develop software in small, manageable cycles called Sprints. It focuses on teamwork, customer feedback, and continuous improvement.

Key Points of Scrum

1. **Sprint:** A time-boxed period of one month or less during which a product increment is developed is known as a Sprint, and a new sprint begins immediately after the previous one ends.
2. **Release:** Usable product increment can be delivered based on requirements, which is referred to as a Release in Scrum.
3. **Sprint Review:** Completed work is presented to stakeholders for feedback and future improvements during the Sprint Review.
4. **Sprint Retrospective:** Continuous improvement in processes and team performance is achieved through the Sprint Retrospective, where the team reflects on the sprint.
5. **Product Backlog:** All features, enhancements, and requirements are organized and prioritized in the Product Backlog.
6. **Sprint Backlog:** Selected items from the Product Backlog, along with a plan to complete them during the sprint, make up the Sprint Backlog.



DSDM (Dynamic System Development Method) :

- DSDM is an Agile method that focuses on delivering software on time and within budget.
- It is based on the idea that users should be involved all the time and that the most important work is done first.

Key Ideas:

- Users are always involved – They give feedback and help the team
- Work is done in steps – The software grows bit by bit
- Time and cost are fixed – Only the most useful features are built first
- Testing is done at every step – Not just at the end

Best for: Projects that must be finished on time and with a fixed budget

CRYSTAL

Crystal is a lightweight and flexible Agile method.

Key Ideas:

- One size doesn't fit all – The method changes based on the project
- Depends on team size and project importance
- Focuses on people, teamwork, and communication
- Less focus on rules, more focus on what works best for the team
- Best for: Small teams with simple or low-risk projects

Agile Unified Process (AUP)

AUP is a simpler version of the traditional Rational Unified Process (RUP), but with Agile ideas.

Key Ideas:

- Uses Agile practices like:
 - Daily meetings
 - Test-first development
 - Continuous feedback
- Breaks the project into phases:
 - Inception – Understand the idea
 - Elaboration – Plan and design
 - Construction – Build the software
 - Transition – Deliver to the user

Best for: Teams who are used to traditional methods and want to move to Agile